

Theory of Computation

303105306

Dr. Sandeep Wadekar

Assistant Professor

Department of Artificial Intelligence & Data Science

UNIT-3

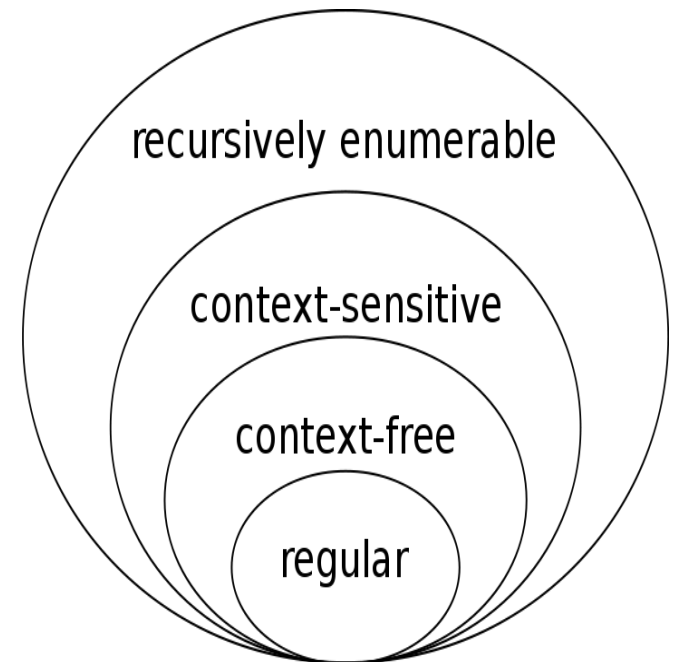
Grammar

Content

Sr. No.	Topic	Page No.
1	Grammar	3
2	Context Free Language (CFL)	4
3	Derivation Tree / Parse Tree	6
4	Ambiguity in CFG	12
5	CFG Simplification	13
6	Chomsky Normal Form (CNF)	18
7	Greibach Normal Form (GNF)	19
8	Pushdown Automata (PDA)	20
9	PDA Equivalence to CFG	23
10	Pumping Lemma of CFG	35
11	Deterministic Pushdown Automata	40
12	Closure Properties of CFLs	43
13	Context Sensitive Grammar and Lang.	58
14	Linear Bounded Automata (LBA)	68

Context Free Language

- CFLs are produced by context free language structure. The arrangement of all setting free dialects is indistinguishable from the set of dialects acknowledged by push down automata, and the set of regular language is a subset of CFG



Context Free Language

- **Grammar:**

- Sentence structure is a lot of rules which check whether a string have a place with a specific language or not. This defined Sentence structure is called Grammar

- Syntax of language is defined by this notation

- Designing of Parser is done by the help of CFG

Derivation Tree/ Parse Tree

- **Generation of Derivation Tree**

- A deduction tree or parse tree is an arranged established tree that graphically speaks to the semantic data a string got from a CFG.

- **Representation Technique**

- Root vertex: Must be labeled by the start symbol.
 - Vertex: Labeled by a non-terminal symbol.
 - Leaves: Labeled by a terminal symbol or ϵ .

Approaches to draw a derivation tree

- **Top-down Approach**

- Starts with the starting symbol S.
- Goes down to tree leaves using productions.

- **Bottom-up Approach**

- Starts from tree leaves.
- Proceeds upward to the root which is the starting symbol S.

.

Derivation of a Tree

- The induction or the yield of a parse tree is the last string acquired by connecting the marks of the leaves of the tree from left to right, overlooking the Nulls. In any case, if all the leaves are Null, deduction is Null

Derivation of a Tree : Example

- Let a CFG $\{N, T, P, S\}$ be $N = \{S\}$, $T = \{a, b\}$, Starting symbol = S , $P = S \rightarrow SS \mid aSb \mid \epsilon$ One derivation from the above CFG is “abaabb”

Answer:-

$S \rightarrow SS$

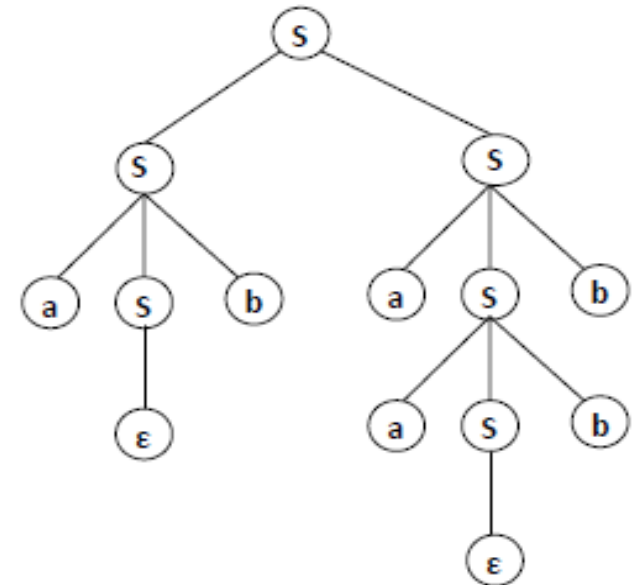
$S \rightarrow aSbS$ (replace $S \rightarrow aSb$)

$S \rightarrow abS$ (replace $S \rightarrow \epsilon$)

$S \rightarrow abaSb$ (replace $S \rightarrow aSb$)

$S \rightarrow abaaSbb$ (replace $S \rightarrow aSb$)

$S \rightarrow abaabb$ (replace $S \rightarrow \epsilon$)



Sentential Form and Partial Derivation Tree

- A fractional deduction tree is a sub-tree of an induction tree/parse tree with the end goal that either the entirety of its kids are in the sub-tree or none of them are in the sub-tree

- Example

In the event that in any CFG the creations/production are:

$$S \rightarrow AB, A \rightarrow aaA \mid \varepsilon, B \rightarrow Bb \mid \varepsilon$$

- The fractional deduction tree can be the accompanying:
 - In the event that an incomplete determination tree contains the root S , it is known as a sentential form.
 - The above example is likewise in sentential form.

Types of Derivation Tree

- Leftmost and Rightmost Derivation of a String
 - Leftmost derivation: A furthest left induction is acquired by applying creation to the furthest left factor in each progression
 - Rightmost derivation: A furthest right deduction is acquired by applying creation to the furthest right factor in each progression.

Left and Right Recursive Grammars

- In a CFG "G", if there is a creation in the structure $X \rightarrow Xa$ where X is a non-terminal and 'a' will be a series of terminals, it is known as a left recursive creation. The syntax having a left recursive creation is known as a left recursive production
- Also, if in a CFG "G", if there is a creation is in the structure $X \rightarrow aX$ where X is a non-terminal and 'a' will be a series of terminals, it is known as a right recursive production. The language structure having a privilege recursive creation is known as a right recursive production.

Ambiguity in Context-Free Grammars

- On the off chance that a setting free sentence structure G has more than one induction tree for some string $w \in L(G)$, it is called an ambiguous grammar. There exist various right-most or left-most determinations for some string produced from that language structure.

CFG Simplification

- In a CFG, it might happen that all the production rules and symbol are not required for the determination of strings. Furthermore, there might be some invalid production and unit creations. End of these production and symbols is called disentanglement of CFGs.

Steps to simply

- Reduction of CFG
- Removal of Unit Productions
- Removal of Null Productions

Reduction of CFG

Derivation of an equivalent grammar, G' , from the CFG, G , such that each variable derives some terminal string.

Derivation Procedure Part 1:

Step 1: Include all symbols, W_1 , that derive some terminal and initialize $i=1$.

Step 2: Include all symbols, W_{i+1} , that derive W_i .

Step 3: Increment i and repeat Step 2, until $W_{i+1} = W_i$.

Step 4: Include all production rules that have W_i in it.

Reduction of CFG

Derivation of an equivalent grammar, G'' , from the CFG, G' such that each symbol appears in a sentential form.

Derivation Procedure part 2:

Step 1: Include the start symbol in Y_1 and initialize $i = 1$.

Step 2: Include all symbols, Y_{i+1} , that can be derived from Y_i and include all production rules that have been applied.

Step 3: Increment i and repeat Step 2, until $Y_{i+1} = Y_i$.

Removal of Unit Productions

Any production rule in the form $A \rightarrow B$ where $A, B \in \text{Non-terminal}$ is called unit production

Removal Procedure:-

Step 1: To remove $A \rightarrow B$, add production $A \rightarrow x$ to the grammar rule whenever $B \rightarrow x$ occurs in the grammar. [$x \in \text{Terminal}$, x can be Null]

Step 2: Delete $A \rightarrow B$ from the grammar.

Step 3: Repeat from step 1 until all unit productions are removed.

Removal of Null Productions

In a CFG, a non-terminal symbol 'A' is a nullable variable if there is a production $A \rightarrow \epsilon$ or there is a derivation that starts at A and finally ends up with ϵ :

$\epsilon: A \rightarrow \dots \rightarrow \epsilon$

Removal Procedure:

Step1 Find out nullable non-terminal variables which derive ϵ .

Step2 For each production $A \rightarrow a$, construct all productions $A \rightarrow x$ where x is obtained from 'a' by removing one or multiple non-terminals from Step 1.

Step3 Combine the original productions with the result of step 2 and remove ϵ -productions.

Chomsky Normal Form

In formal language theory, a CFG "G " is said to be in Chomsky normal form if all of its production rules are of the form:

$Q \rightarrow RE,$
or $Q \rightarrow f,$
or $S \rightarrow \epsilon,$

Here Q,R,E are Non terminals and S is starting symbol. f and ϵ (epsilon) are terminals where ϵ represent NULL

Greibach Normal Form

In formal language theory, a CFG is in Greibach normal form (GNF) if the right-hand sides of all production rules start with a terminal symbol, optionally followed by some variables.

Either that in the form:

$$P \rightarrow aQWERTY$$

$$P \rightarrow a$$

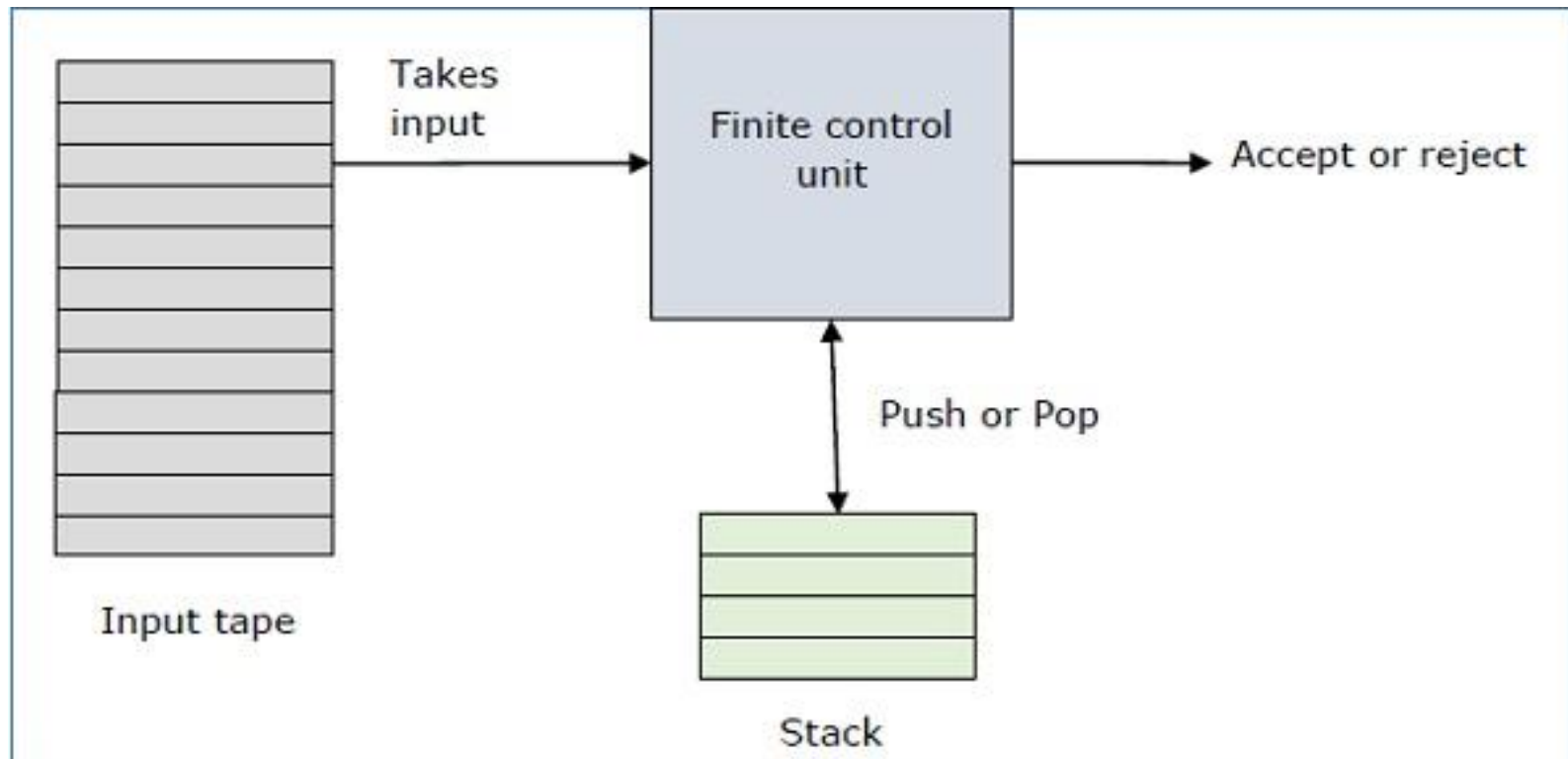
$$S \rightarrow \epsilon$$

Here P,Q,W,E,R,T,Y are non terminals and S is start symbol. ϵ and a are terminals

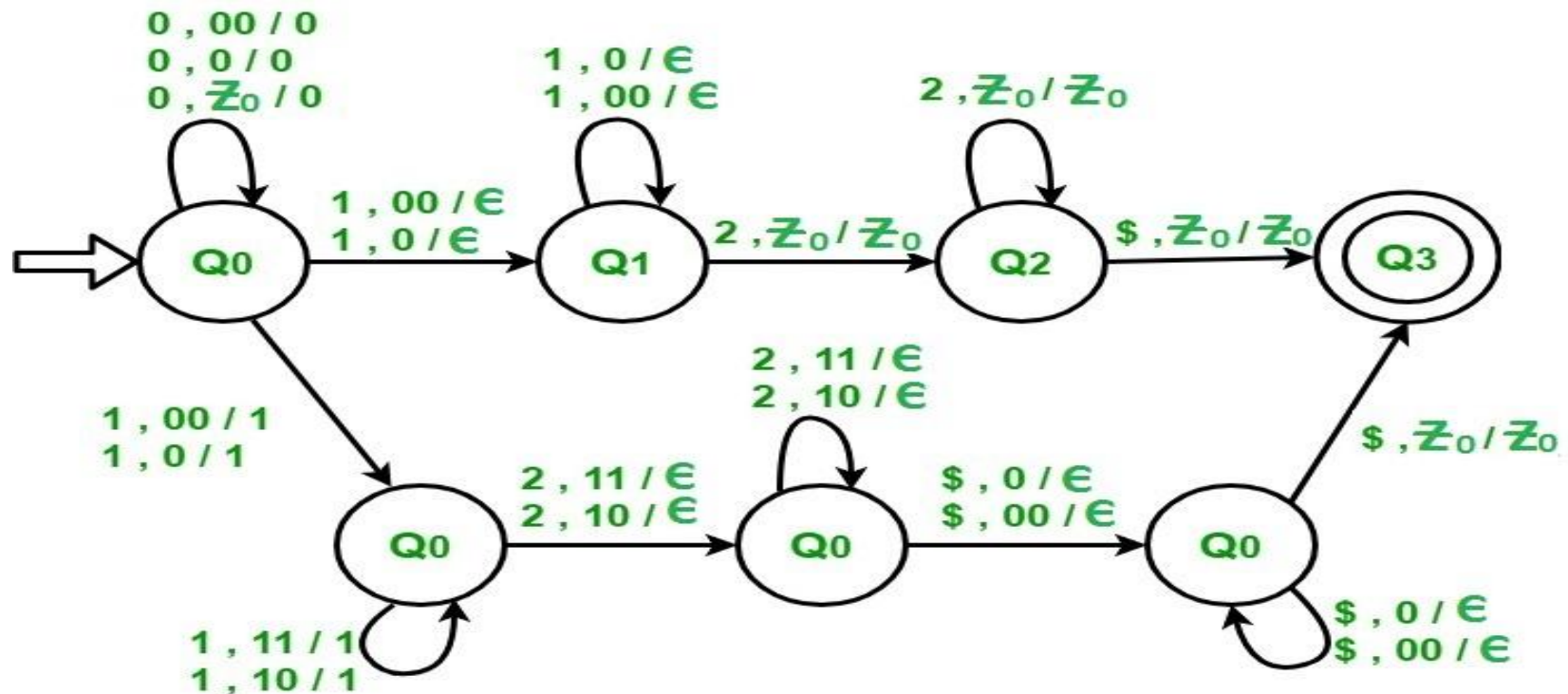
Non Deterministic Push down Automata

- A nondeterministic pushdown automaton (npda) is basically an nfa with a stack added to it.
- A nondeterministic pushdown automaton or npda is a 7-tuple
$$M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$$
- Q is a finite set of states,
 Σ is the input alphabet,
 Γ is the stack alphabet,
 δ is a transition function,
 $q_0 \in Q$ is the initial state,
 $z \in \Gamma$ is the stack start symbol, and
 $F \subseteq Q$ is a set of final states.

Block diagram of PDA



Example of NPDA



PDA equivalence to CFG

- Algorithm to find PDA corresponding to a given CFG:

Input – A CFG, $G = (V, T, P, S)$

Output – Equivalent PDA, $P = (Q, \Sigma, S, \delta, q_0, I, F)$

Step 1 – Convert the productions of the CFG into GNF.

Step 2 – The PDA will have only one state $\{q\}$.

Step 3 – The start symbol of CFG will be the start symbol in the PDA.

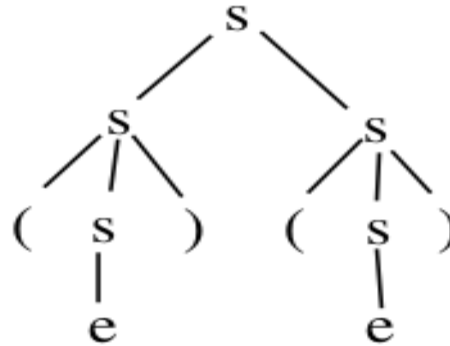
Step 4 – All non-terminals of the CFG will be the stack symbols of the PDA and all the terminals of the CFG will be the input symbols of the PDA.

Step 5 – For each production in the form $A \rightarrow aX$ where a is terminal and A, X are combination of terminal and non-terminals, make a transition $\delta(q, a, A)$

What is Parse Tree??

Parse trees are a representation of derivations that is much more compact. Several derivations may correspond to the same parse tree. For example, in the balanced parenthesis grammar, the following parse tree: HC

What is Parse Tree??



Corresponds to the derivation $S \Rightarrow SS \Rightarrow S(S) \Rightarrow (S)(S) \Rightarrow (S)() \Rightarrow ()()$

as well as this one:

$S \Rightarrow SS \Rightarrow (S)S \Rightarrow (S)(S) \Rightarrow ()(S) \Rightarrow ()()$

Parse Tree

- In a parse tree, the points are called nodes. Each node has a label on it.
- The topmost node is called the root. The bottom nodes are called leaves.
- In a parse tree for a grammar G , the leaves must be labelled with terminal symbols from G , or with $\varnothing$. The root is often labeled with the start symbol of G , but not always.
- If a node N labeled with A has children $N_1, N_2, \dots, N_k$ from left to right, labeled with $A_1, A_2, \dots, A_k$, respectively, then $A \rightarrow A_1A_2, \dots A_k$ must be a production in the grammar G .

Parse Tree

- The yield of a parse tree is the concatenation of the labels of the leaves, from left to right. The yield of the tree above is $()()$.

- **Leftmost and Rightmost Derivations**

- In a leftmost derivation, at each step the leftmost nonterminal is replaced. In a rightmost derivation, at each step the rightmost nonterminal is replaced.
- Such replacements are indicated by $\underline{R}$ and $\underline{L}$ respectively.
- Their transitive closures are $\underline{R}^*$ and $\underline{L}^*$ respectively.

Parse Tree

- In the balanced parenthesis grammar, this is a leftmost derivation:

$S \Rightarrow SS \Rightarrow (S)S \Rightarrow ()S \Rightarrow ()(S) \Rightarrow ()()$.

This is a rightmost derivation:

$S \Rightarrow SS \Rightarrow S(S) \Rightarrow S() \Rightarrow (S)() \Rightarrow ()()$

Ambiguity In CFG

- A context-free grammar $G = (V, \Sigma, R, S)$ is ambiguous if there is some string $w \in \Sigma^*$ such that there are two distinct parse trees T_1 and T_2 having S at the root and having yield w .
- Equivalently, w has two or more leftmost derivations, or two or more rightmost derivations.
- Note that languages are not ambiguous; grammars are. Also, it has to be the same string w with two different (leftmost or rightmost) derivations for a grammar to be ambiguous.

Ambiguity In CFG

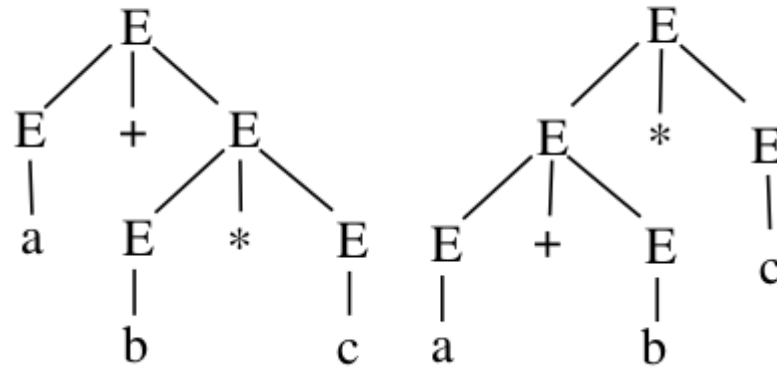
- Here is an example of an ambiguous grammar:
- $E \rightarrow E + E$
- $E \rightarrow E * E$
- $E \rightarrow (E)$
- $E \rightarrow a$
- $E \rightarrow b$
- $E \rightarrow c$

Ambiguity In CFG

- In this grammar, the string $a + b * c$ can be parsed in two different ways corresponding to doing the addition before or after the multiplication. This is very bad for a compiler, because the compiler uses the parse tree to generate code, meaning that this string could have two very different semantics. Here are two parse trees for the string $a + b * c$ in this grammar:

Here are two parse trees for the string $a + b * c$ in this grammar:

Ambiguity In CFG



Ambiguity In CFG

- There is a notion of inherent ambiguity for context-free languages; a context-free language L is inherently ambiguous if every context-free grammar G for L is ambiguous. As an example, the language $\{a^n b^n c^m d^m : n \geq 1, m \geq 1\} \cup \{a^n b^m c^m d^n : n \geq 1, m \geq 1\}$ is inherently ambiguous. In any context-free grammar for L , some strings of the form $a^n b^n c^n d^n$ will have two distinct parse trees.
- Unfortunately, the problem of whether a context-free grammar is ambiguous, is undecidable. However, there are some patterns in a context-free grammar that frequently indicate ambiguity:

Ambiguity In CFG

- $S \rightarrow SS$
- $S \rightarrow a$
- $S \rightarrow A$
- $A \rightarrow AA$
- $A \rightarrow a$
- $S \rightarrow AA$
- $A \rightarrow S$
- $A \rightarrow a$
- $S \rightarrow SbS$

Ambiguity in CFG

- $S \rightarrow a$
- $S \rightarrow AbA$
- $A \rightarrow S$
- $A \rightarrow a$

Ambiguity In CFG

The following is not ambiguous:

$$S \rightarrow aS$$

$$S \rightarrow bS$$

$$S \rightarrow \epsilon$$

In general, a production $A \rightarrow AA$ causes ambiguity if it is reachable from the start symbol and some terminal string is derivable from A .

Pumping Lemma of CFG

- For every CFL L there is a constant $k \geq 0$ such that for any word z in L of length at least k , there are strings u, v, w, x, y such that
- $z = uvwxy$,
- $\forall x \neq \epsilon$
- $|vwx| \leq k$,
- and for each $i \geq 0$, the string uv^iwx^iy belongs to L .



Parse trees for CFG's

Derivations can be represented as parse trees:

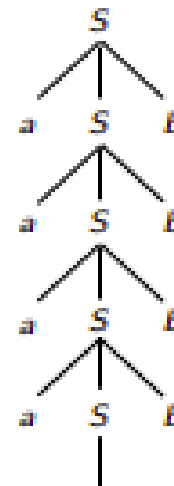
CFG G_2

$S \rightarrow aSb$

$S \rightarrow \epsilon.$

Example derivation:

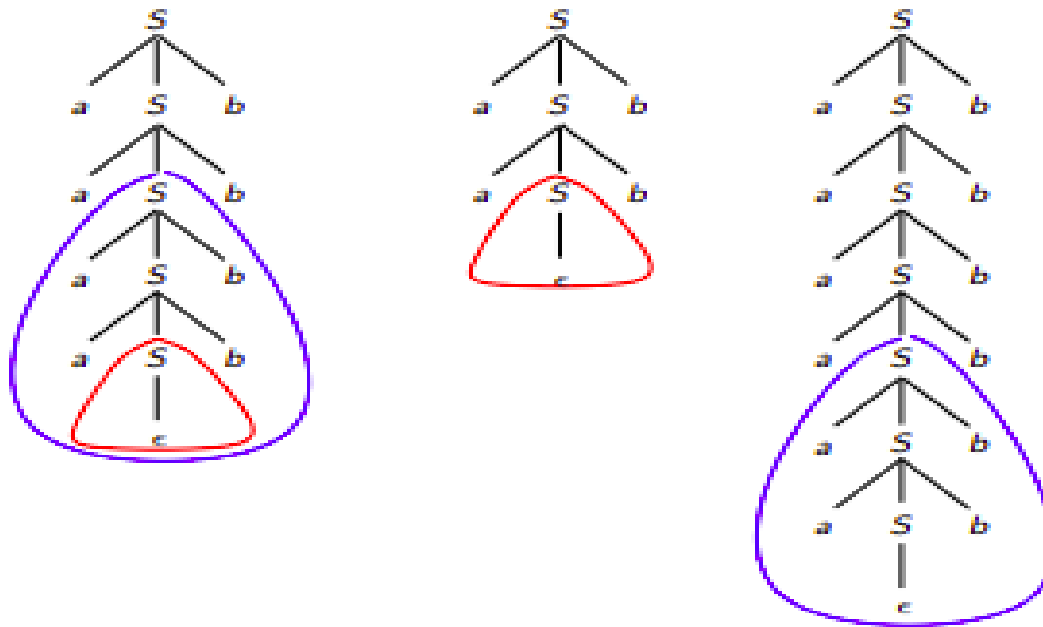
$S \Rightarrow aSb$
 $\Rightarrow aaSbb$
 $\Rightarrow aaaSbbb$
 $\Rightarrow aaaaSbbbb$
 $\Rightarrow aaaabbbb.$



Parse trees for CFG's

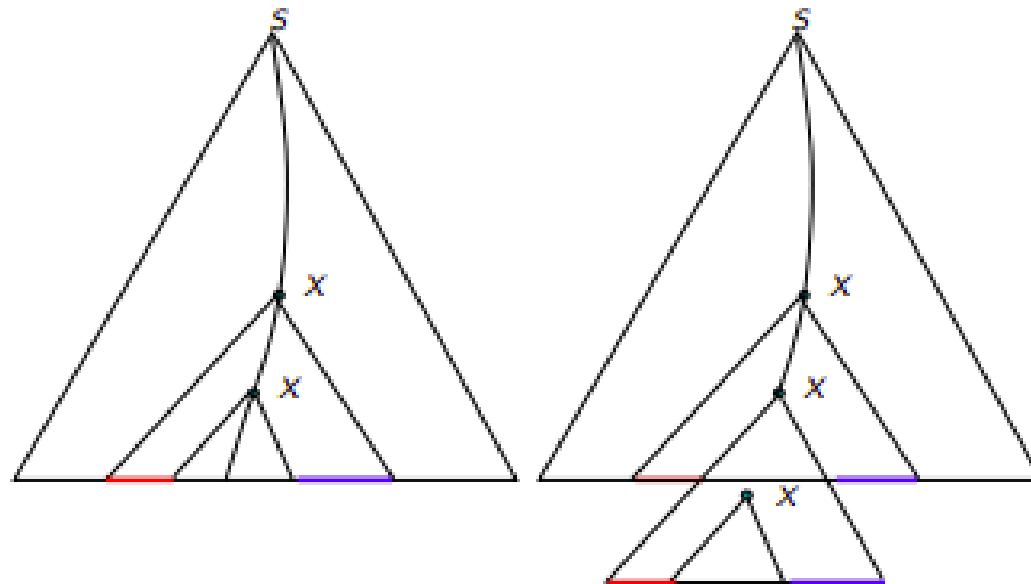
- **Cutting and pasting in parse trees**

Subtrees hanging at same non-terminal can be replaced for each other.



Proof idea

- A long string must have a deep parse tree, which in turn means a path with a repeated non-terminal.

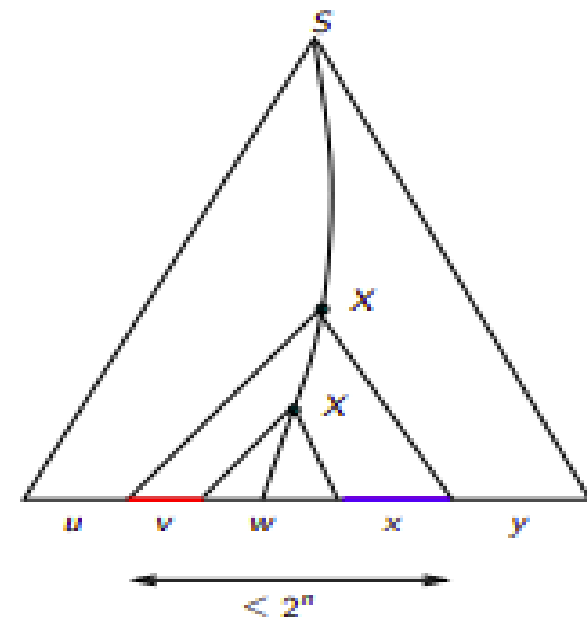


Proof

- Let G be a CNF grammar for L .
- A complete binary tree with i levels has 2^{i-1} leaf nodes.
- A parse tree in G with i levels has a terminal string ("yield") of length at most 2^{i-2} .
- Hence a string of length 2^n or more, must have a parse tree of at least $n + 2$ levels.
- Take $k = 2^n$ where n is number of non-terminals in G .

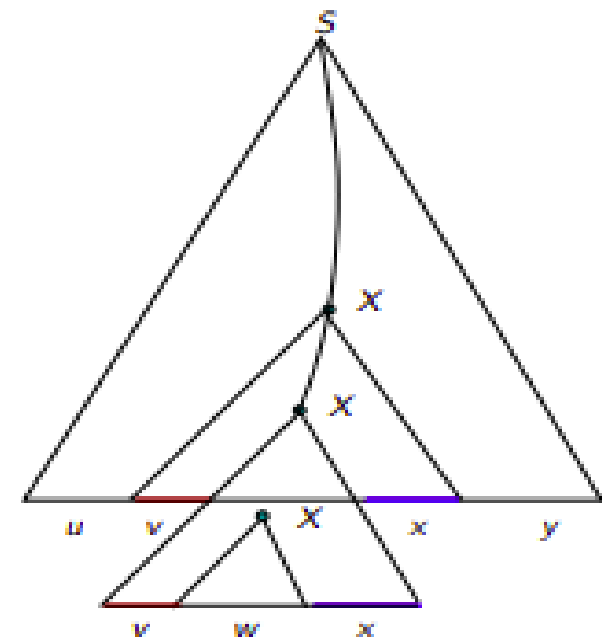
Proof II

- Consider parse tree in G of a string z of length at least $k = 2^n$.
- Consider **longest path** from root to leaf.
- Choose the first repeated non-terminal X starting from bottom of path.
- Path from upper X down to leaf is at most $n + 2$ levels. Also it must be the **longest** path in the subtree rooted at X . Hence length of vw is at most 2^n .
- Also $vx \neq \epsilon$, as G is a CNF grammar.



Proof II

- Consider parse tree in G of a string z of length at least $k = 2^n$.
- Consider **longest path** from root to leaf.
- Choose the first repeated non-terminal X starting from bottom of path.
- Path from upper X down to leaf is at most $n + 2$ levels. Also it must be the **longest** path in the subtree rooted at X . Hence length of vx is at most 2^n .
- Also $vx \neq \epsilon$, as G is a CNF grammar.
- Each uv^iwx^iy also belongs to $L(G)$.



Deterministic Push Down Automata

Two transitions $((p_1, u_1, \beta_1), (q_1, \gamma_1))$ and $((p_2, u_2, \beta_2), (q_2, \gamma_2))$ are compatible if

1. $p_1 = p_2$,
2. u_1 and u_2 are compatible (which means that u_1 is a prefix of u_2 or that u_2 is a prefix of u_1),
3. β_1 and β_2 are compatible.

Deterministic Push Down Automata

- A pushdown automaton is deterministic if for every pair of compatible transitions, these transitions are identical.
- Let L be a language defined over the alphabet Σ , the language L is deterministic context-free if and only if it is accepted by a deterministic pushdown automaton.

Deterministic Push Down Automata

- Not all context-free languages are deterministic context-free.
- $L_1 = \{wcw^R \mid w \in \{a, b\}^*\}$ is deterministic context-free.
- $L_2 = \{ww^R \mid w \in \{a, b\}^*\}$ is context-free, but not deterministic context-free.
- Not all context-free languages are deterministic context-free.
- $L_1 = \{wcw^R \mid w \in \{a, b\}^*\}$ is deterministic context-free.
- $L_2 = \{ww^R \mid w \in \{a, b\}^*\}$ is context-free, but not deterministic context-free.

Closure properties of CFLs

- **CFLs are closed under**
 - Union
 - Concatenation
 - Kleene Star
 - Reversal
 - Homomorphism
- **Regular Languages closed under**
 - Intersection
 - Difference
 - Complementation
- **CFLs are not closed under intersection, difference, or complementation, why?**

Union

- If L_1 and L_2 are two context free languages, their union $L_1 \cup L_2$ will also be context free.
- Let L_1 and L_2 be CFLs with grammars G_1 and G_2 , respectively.
- Assume G_1 and G_2 have no variables in common.
- Let S_1 and S_2 be the start symbols of G_1 and G_2 .
- Form a grammar for $(L_1 \cup L_2)$ by combining all the symbols and productions of G_1 and G_2 .
- Then, add a new start symbol S . Add productions $S \rightarrow S_1 \mid S_2$.

Union

- **Example:**

$$L1 = \{ a^n b^n c^m \mid m \geq 0 \text{ and } n \geq 0 \}$$

$$L2 = \{ a^n b^m c^m \mid n \geq 0 \text{ and } m \geq 0 \}$$

$$L3 = L1 \cup L2 = \{ a^n b^n c^m \cup a^n b^m c^m \mid n \geq 0, m \geq 0 \} \text{ is also context free.}$$

- Here Language L1 generates all strings that contains occurrence of a's equals to occurrence of b's and L2 generates all strings that contains occurrence of b's equals to the occurrence of c's.
- Union require either of two condition require to be true. It can be accepted by pushdown automata.
- So, Language L3 is also CFL.

Concatenation

- L_1 and L_2 are CFLs, then their concatenation $L_1.L_2$ will also be context free
- Let L_1 and L_2 be CFL's with grammars G_1 and G_2 , respectively.
- Assume G_1 and G_2 have no variables in common.
- Let S_1 and S_2 be the start symbols of G_1 and G_2 .
- Form a new grammar for $L_1.L_2$ by starting with all symbols and productions of G_1 and G_2 .
- Add a new start symbol S and production $S \rightarrow S_1S_2$.
- Every derivation from S results in a string in L_1 followed by one in L_2 .

Concatenation

- **Example :**

$$L1 = \{ a^n b^n \mid n \geq 0 \}$$

$$L2 = \{ c^m d^m \mid m \geq 0 \}$$

$$L3 = L1.L2 = \{ a^n b^n c^m d^m \mid m \geq 0 \text{ and } n \geq 0 \}$$
 is also context free.

- Here L1 grammar generate all strings which contains equal occurrence of a's and b's and L2 generate equal occurrence of c's and d's. Language L3 can be accepted by Pushdown automata.
- Hence, Concatenation is closed under CFLs.

Kleene Star

- L_1 is context free, then its Kleene closure L_1^* will also be context free.
- Let L have grammar G , with start symbol S_1 .
- Grammar for L^* by introducing to G a new start symbol S and the productions
 $S \rightarrow S_1 S \mid \epsilon$.
- A rightmost derivation from S generates a sequence of zero or more S_1 's, each of which generates some string in L .

- **Example :**

$$L_1 = \{ a^n b^n \mid n \geq 0 \}$$

$$L_1^* = \{ a^n b^n \mid n \geq 0 \}^* \text{ is also context free.}$$

Reversal

- L is a CFL with grammar G
- Grammar for L_R by reversing the right side of every production.

- **Example:**

Let G have $S \rightarrow 0S1 \mid 01$.

$S \rightarrow 1S0 \mid 10$.

- L_R is also Context free grammar.

Homomorphism

- Let L be a CFL with grammar G .
- Let h be a homomorphism on the terminal symbols of G .
- Construct a grammar for $h(L)$ by replacing each terminal symbol a by $h(a)$.

- **Example:**

G has productions $S \rightarrow 0S1 \mid 01$.

h is defined by $h(0) = ab$, $h(1) = \epsilon$.

$h(L(G))$ has the grammar with productions $S \rightarrow abS \mid ab$.

Non-Closure Properties of CFL's - Intersection

- Let L_1 and L_2 are two context free languages

$$L_1 = \{ a^n b^n c^m \mid n \geq 0 \text{ and } m \geq 0 \}$$

$$L_2 = \{ a^m b^n c^n \mid n \geq 0 \text{ and } m \geq 0 \}$$

$$L_3 = L_1 \cap L_2 = \{ a^n b^n c^n \mid n \geq 0 \}$$

- L_1 generate all strings of number of a's should be equal to number of b's and
 L_2 generate all strings of number of b's should be equal to number of c's.
- Intersection require both conditions need to be true
- It cannot be accepted by pushdown automata, so it is not context free.

Complementation

- Let L_1 and L_2 are two context free languages

$$L_1 = \{ a^n b^n c^m \mid n \geq 0 \text{ and } m \geq 0 \}$$

$$L_2 = \{ a^m b^n c^n \mid n \geq 0 \text{ and } m \geq 0 \}$$

$$\overline{L_1 \cup L_2} = L_1 \cap L_2 = \{ a^n b^n c^n \}$$

- CFLs are not closed under intersection.
- Language is not context free and it can not accepted by Pushdown automata.
- Thus, CFLs are not closed under Complementation.

Difference

- Let L_1 and L_2 are two context free languages
- Proof: $L_1 \cap L_2 = L_1 - (L_1 - L_2)$.
- CFLs are not closed under Intersection.
- If CFLs were closed under difference, they require to be closed under intersection, but they are not.
- Thus, CFLs are not closed under difference

Context Sensitive Grammar and Languages

- **Hierarchy of languages.**

- Type-0 : Recursively enumerable language
- Type-1 : Context Sensitive language
- Type-2 : Context Free language
- Type-3 : Regular language

- We discuss Context Sensitive Language and corresponding state machine, (Linear Bounded Automaton(LBA)), equivalence and properties of Context Sensitive Languages.

Definition : Context Sensitive Grammar(CSG)

- Context Sensitive Grammar (CSG) is a quadruple $G=(N,\Sigma,P,S)$ where,
 - N is set of non-terminal symbols
 - Σ is set of terminal symbols
 - S is set of start symbol
 - P is set of production in form of $\alpha A \beta \rightarrow \alpha \gamma \beta$ where $\gamma \neq \epsilon$
- Derivation non-terminal A will be changed to γ only
- CSG is Non-contracting grammar as $\gamma \neq \epsilon$ then $\alpha \rightarrow \beta \Rightarrow |\alpha| \leq |\beta|$

Context Sensitive Language(CSL)

- The language that can be defined by context-sensitive grammar is called Context sensitive language(CSL).

- Example:**

Consider the following CSG.

$$S \rightarrow abc/aAbc$$

$$Ab \rightarrow bA$$

$$Ac \rightarrow Bbcc$$

$$bB \rightarrow Bb$$

$$aB \rightarrow aa/aaA$$

- Derivation of CSL**

$$S \rightarrow aAbc$$

$$\rightarrow abAc$$

$$\rightarrow abBbcc$$

$$\rightarrow aBbbcc$$

$$\rightarrow aaAbbcc$$

$$\rightarrow aabAbcc$$

$$\rightarrow aabbAcc$$

$$\rightarrow aabbBbcc$$

$$\rightarrow aabBbbcc$$

$$\rightarrow aaBbbbcc$$

$$\rightarrow aaabbbcc$$

Context sensitive language

$$L = \{a^n b^n c^n \mid n \geq 1\}.$$

Closure properties of CSLs

- Union
- Intersection
- Complement
- Concatenation
- Kleene closure
- Reversal

Equivalence of CSL

- The following grammar(G) is context-sensitive.

$S \rightarrow aTb \mid ab$

$aT \rightarrow aaTb \mid ac$

- The language generated by grammar G

$$L(G) = \{ab\} \cup \{a^n cb^n \mid n > 0\}$$

- Language is also a context-free.
- For example, Context free grammar(G1) for this.

$S \rightarrow aTb \mid ab$

$T \rightarrow aTb \mid c$

- Any context-free language is context sensitive.
- Not all context sensitive but it need not be context free.

Equivalence of CSL

Theorem: Every context-sensitive language L is recursive.

- Let L be CSL, G be CSG
- Derivation of string w , $S \rightarrow S_1, S_1 \rightarrow S_2, S_2 \rightarrow S_3, \dots = w$
- No of steps are Bound on possible derivations. We know that $|x_i| < |x_{i+1}|$ (G is non-contracting).
- Check whether w is in $L(G)$ as follows
 - Construct a transition graph whose vertices are the strings of length $|w|$.
 - To find Paths correspond to derivation in G . Add edge from x to y if $x \rightarrow y$
 - $w \in L(G)$ if there is a path from S to w .

Equivalence of CSL

Theorem: There exists some recursive language that is not context sensitive.

- **Language L is recursive**

- Create possible CSG $G_i = (N_i, \{0; 1; 2; 3; 4; 5; 6; 7; 8; 9\}, S_i, P_i)$ which generates numbers.
- We can define language L, which contains the numbers of the grammars which does not generate the number of its position in the list.

$$L = \{i \mid i \notin L(G_i)\}.$$

- We can create a list of all context-sensitive generative grammars which generates numbers, and we can decide whether or not a context-sensitive grammar generates its position in the list.
- Hence language L is recursive.

Equivalence of CSL

Theorem: There exists a recursive language that is not context sensitive.

- **Language L is not context sensitive**

- Assume that L is a CSL

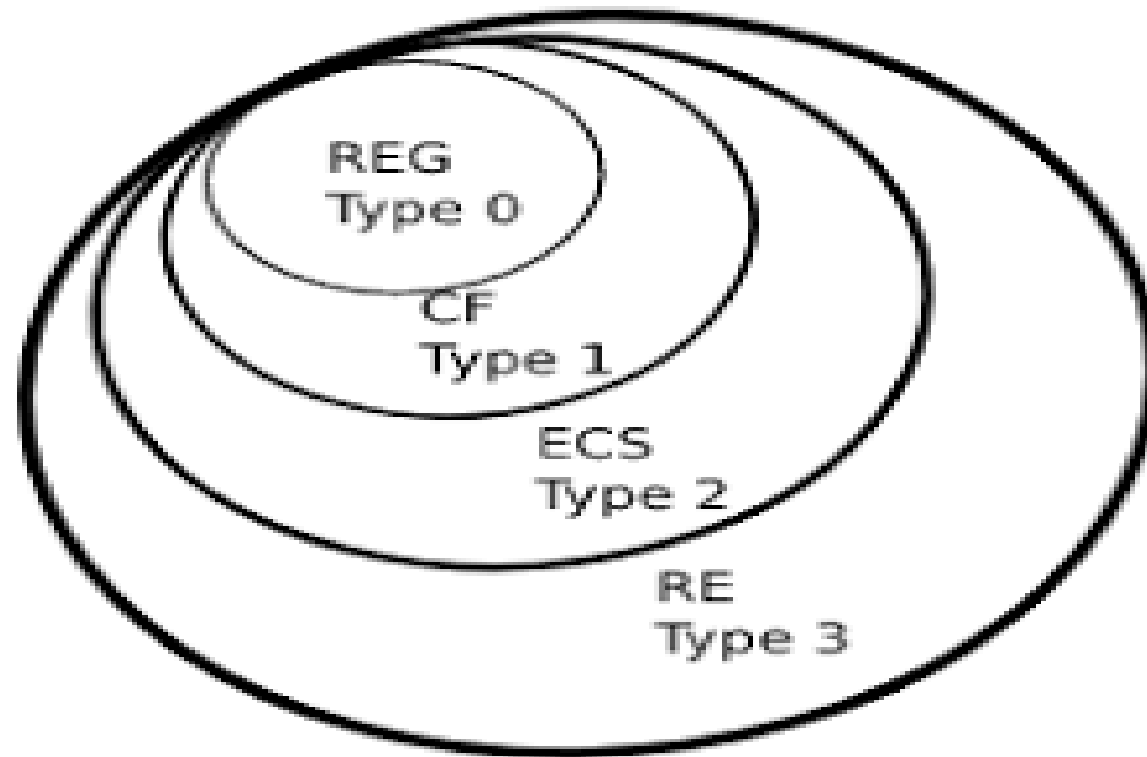
- So there is a CSG G_c , s.t $L(G_c) = L$ for some c .

- If $c \in L(G_c)$, by the definition of L, we have $c \notin L$, but $L = L(G_c)$. So a contradiction.

- If $c \notin L(G_c)$, then $c \in L$ is also a contradiction since $L = L(G_c)$.

- Hence language L is not context sensitive.

Chomsky Hierarchy



Chomsky Hierarchy

Type	Language	Automaton	Production rules
Type 0 Unrestricted	Recursively enumerable	Turing Machine	$\alpha \rightarrow \beta$
Type 1 Context-sensitive	Context-sensitive	Linear-bounded automaton	$\alpha A \beta \rightarrow \alpha \gamma \beta$
Type 2 Context-free	Context-free	Pushdown automaton	$A \rightarrow \gamma$
Type 3 Regular	Regular	Finite state automaton	$A \rightarrow a$ and $A \rightarrow aB$

Linear Bounded Automata

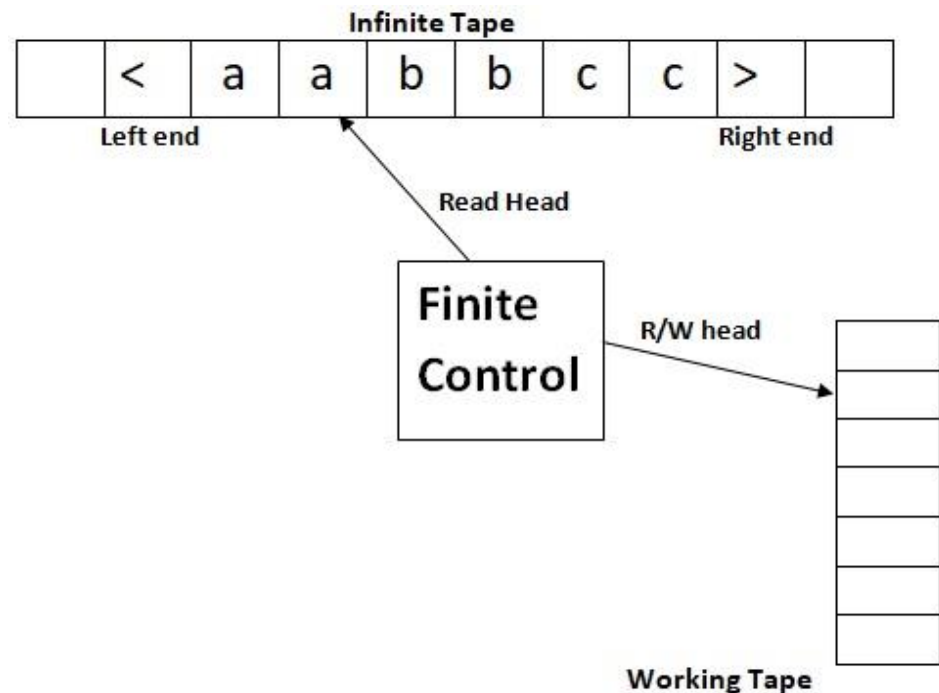
- Linear Bounded Automata is a single tape non-deterministic Turing Machine with two special tape symbols call them left marker $<$ and right marker $>$.
- The transitions should satisfy these conditions:
 - It should not replace the marker symbols by any other symbol.
 - It should not write on cells beyond the marker symbols.
- Configuration will be: $\langle q_0 a_1 a_2 a_3 a_4 a_5 \dots a_n \rangle = \langle q_0 w \rangle$

Linear Bounded Automata

- Linear Bounded Automata is a non-deterministic Turing Machine,

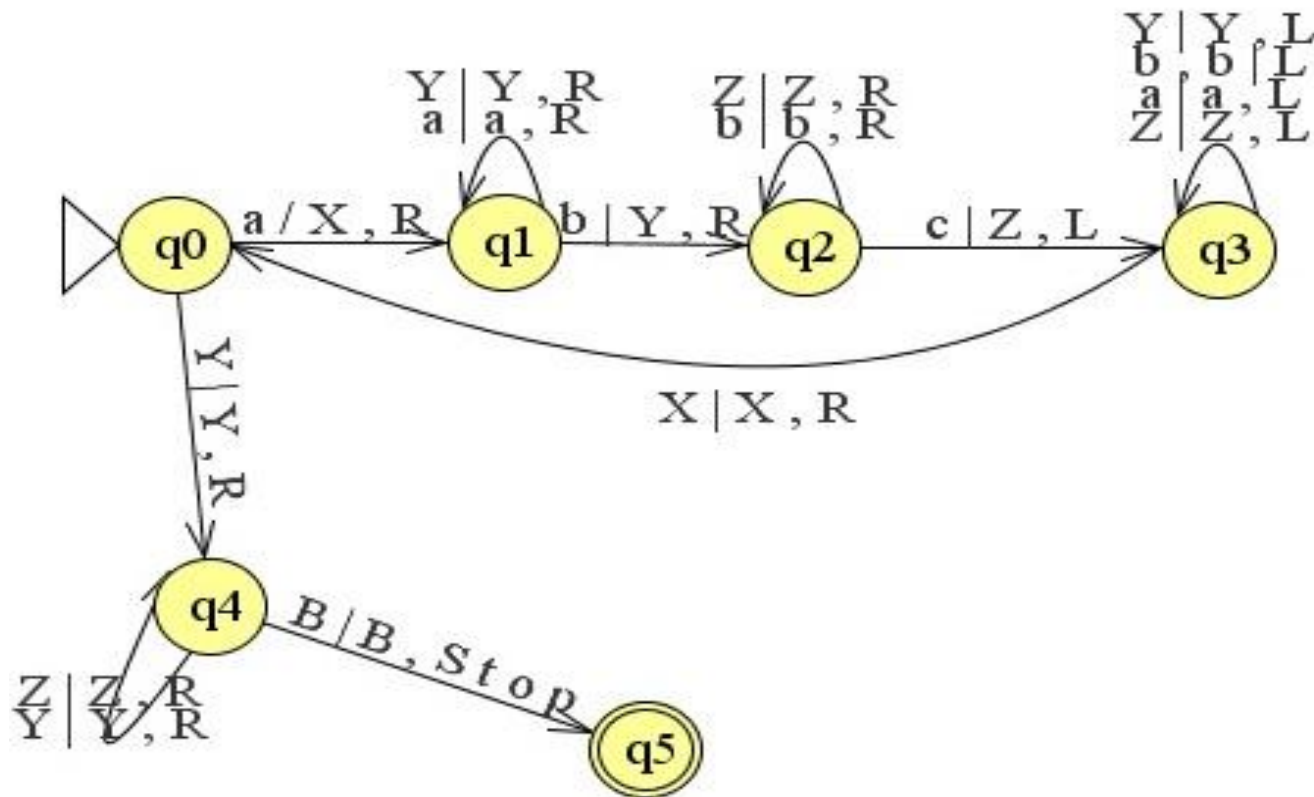
$M = (Q, \Sigma, T, \delta, B, F, q_0, <, >, t, r)$ Where,

- Q is set of all states
- Σ is set of all terminals
- T is set of all tape alphabets
- δ is set of transitions
- B is blank symbol
- q_0 is the initial state
- $<$ is left marker and $>$ is right marker
- t is accept state
- r is reject state



Linear Bounded Automata

- Turing Machine for Context sensitive language $L = \{a^n b^n c^n \mid n \geq 1\}$.



References

1. **Introduction to Automata theory, languages and Computation** By John E. Hopcroft, Rajiv Motwani and Jeffery D. Ullman | Pearson
2. **Elements of the Theory of Computation** Harry R. Lewis and Christos H. Papadimitriou | Pearson Education Asia
3. **Introduction to the Theory of Computation** Michael Sipser | PWS Publishing
4. **Introduction to Languages and the Theory of Computation** John C. Martin | McGraw Hill
5. **Automata and Computability** Dexter C. Kozen | Springer

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

